

Lab 2 — Task 3: Docker Storage

Akash Singh Sagar | Repository: github.com/akashsinghsagar/devops-lab2

Objective

Compare three Docker storage strategies on the same application image: no persistent storage, a named volume, and a bind mount — observing what happens to the visit counter each time a container is removed and recreated.

Step 1 — No persistent storage (baseline)

A container is run with no volume attached. Three requests correctly increment the in-container counter file to 3, and docker exec confirms /data/counter.txt exists while the container is alive. The container is then removed and a fresh container (lab2-nostorage-2) is started from the same image.

```
docker run -d -p 5002:5000 --name lab2-nostorage lab2-python-app:1.0
curl.exe -s http://localhost:5002/
curl.exe -s http://localhost:5002/
curl.exe -s http://localhost:5002/
docker exec lab2-nostorage cat /data/counter.txt
docker stop lab2-nostorage
docker rm lab2-nostorage
docker run -d -p 5002:5000 --name lab2-nostorage-2 lab2-python-app:1.0
curl.exe -s http://localhost:5002/
```

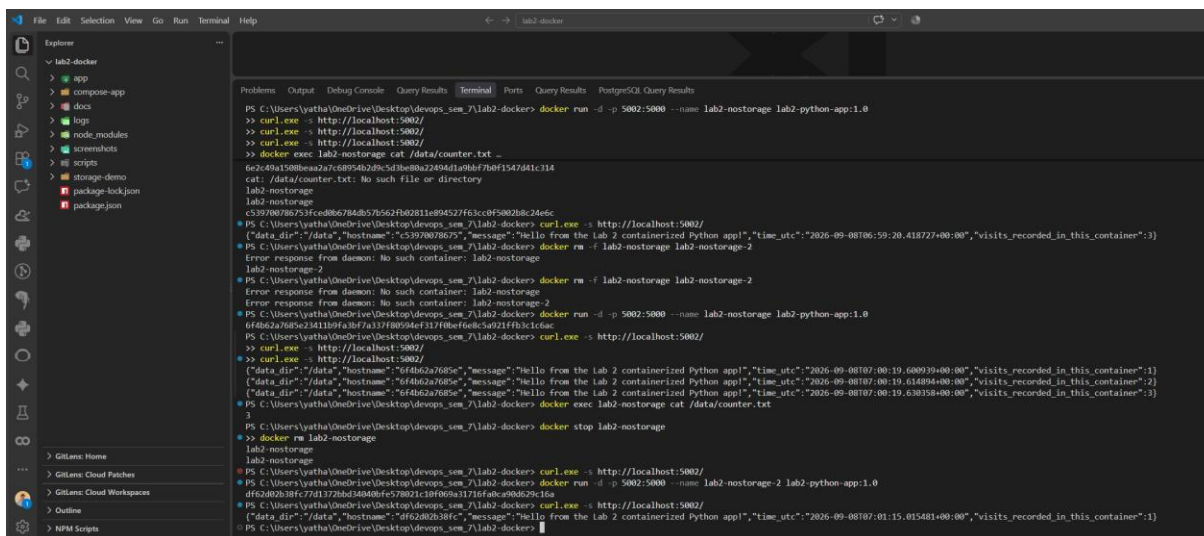


Figure 1 — `visits_recorded_in_this_container` climbs 1 → 2 → 3 and `docker exec` confirms the value on disk, but the replacement container (lab2-nostorage-2) starts back at 1.

Step 2 — Named volume

A named volume (lab2-data-volume) is created and mounted at /data. A container (lab2-vol-1) writes to it, is then stopped and removed, and a second container (lab2-vol-2) is started with the same volume attached.

```
docker volume create lab2-data-volume
docker volume inspect lab2-data-volume
docker run -d -p 5004:5000 --name lab2-vol-1 -v lab2-data-volume:/data lab2-python-app:1.0
curl.exe -s http://localhost:5004/
curl.exe -s http://localhost:5004/
```

```

curl.exe -s http://localhost:5004/
docker stop lab2-vol-1
docker rm lab2-vol-1
docker volume ls --filter name=lab2-data-volume
docker run -d -p 5004:5000 --name lab2-vol-2 -v lab2-data-volume:/data lab2-python-app:1.0
curl.exe -s http://localhost:5004/

```

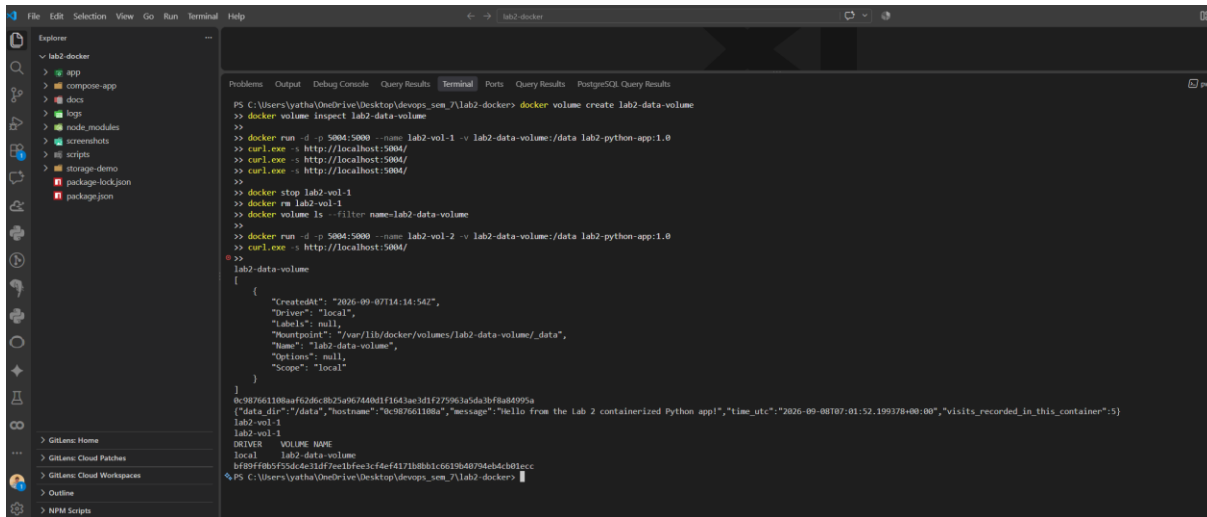


Figure 2 — `docker volume inspect` shows the volume backed by `/var/lib/docker/volumes/lab2-data-volume/_data`; after `lab2-vol-1` is removed, `lab2-vol-2` continues the count instead of resetting to 1.

Step 3 — Bind mount

A host folder (`storage-demo\bind-data`) is bind-mounted into a container at `/data`. The counter file is inspected directly from the Windows host with `type`, the container is removed, and a second container is pointed at the same host folder.

```

mkdir storage-demo\bind-data
docker run -d -p 5005:5000 --name lab2-bind-1 -v "${PWD}\storage-demo\bind-data:/data"
lab2-python-app:1.0
curl.exe -s http://localhost:5005/
curl.exe -s http://localhost:5005/
type storage-demo\bind-data\counter.txt
docker stop lab2-bind-1
docker rm lab2-bind-1
dir storage-demo\bind-data\
docker run -d -p 5005:5000 --name lab2-bind-2 -v "${PWD}\storage-demo\bind-data:/data"
lab2-python-app:1.0
curl.exe -s http://localhost:5005/

```

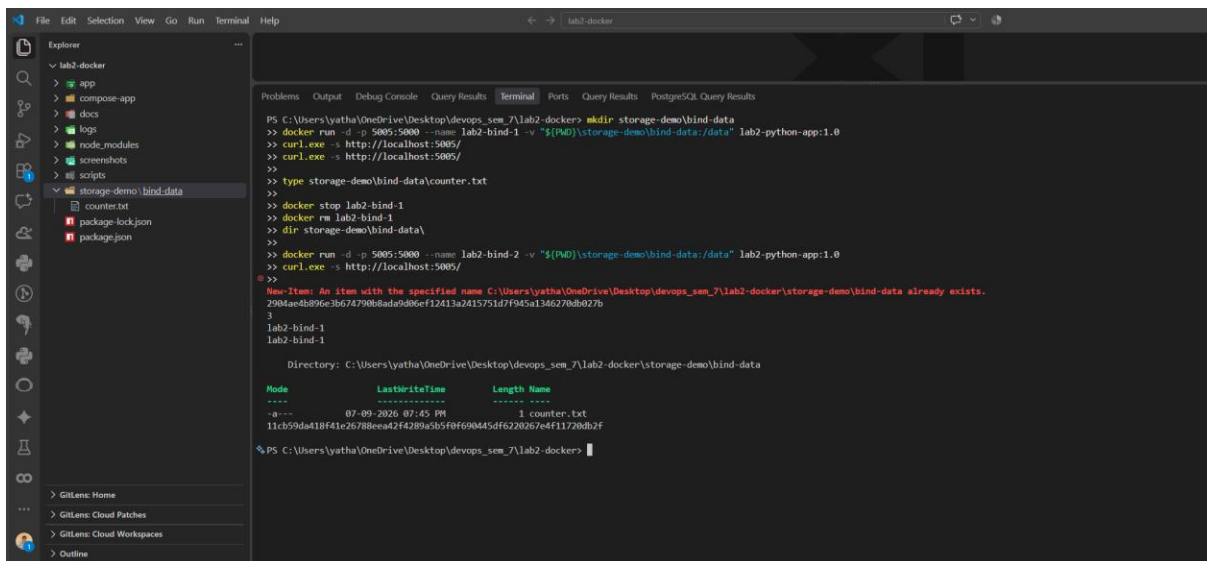


Figure 3 — counter.txt is visible and editable directly on the Windows filesystem (storage-demo\bind-data\); a replacement container mounted at the same path picks up the existing value.

Result

Three storage models were compared side by side on the same image. Without a volume, all writes lived only in the container's writable layer and vanished the moment the container was removed. A named volume and a bind mount both kept /data outside the container's lifecycle: a named volume persisted the counter across container removal/recreation while remaining fully Docker-managed, and a bind mount additionally exposed the same file directly on the host filesystem for outside inspection and editing. Task 3 is complete.